

2019-2020学年秋季学期

漏洞利用与攻防实践

*Exploiting Software Vulnerability-
Techniques and Practice*

小组成员：刘心宇 井宇
陈晓惠

漏洞利用与攻防实践

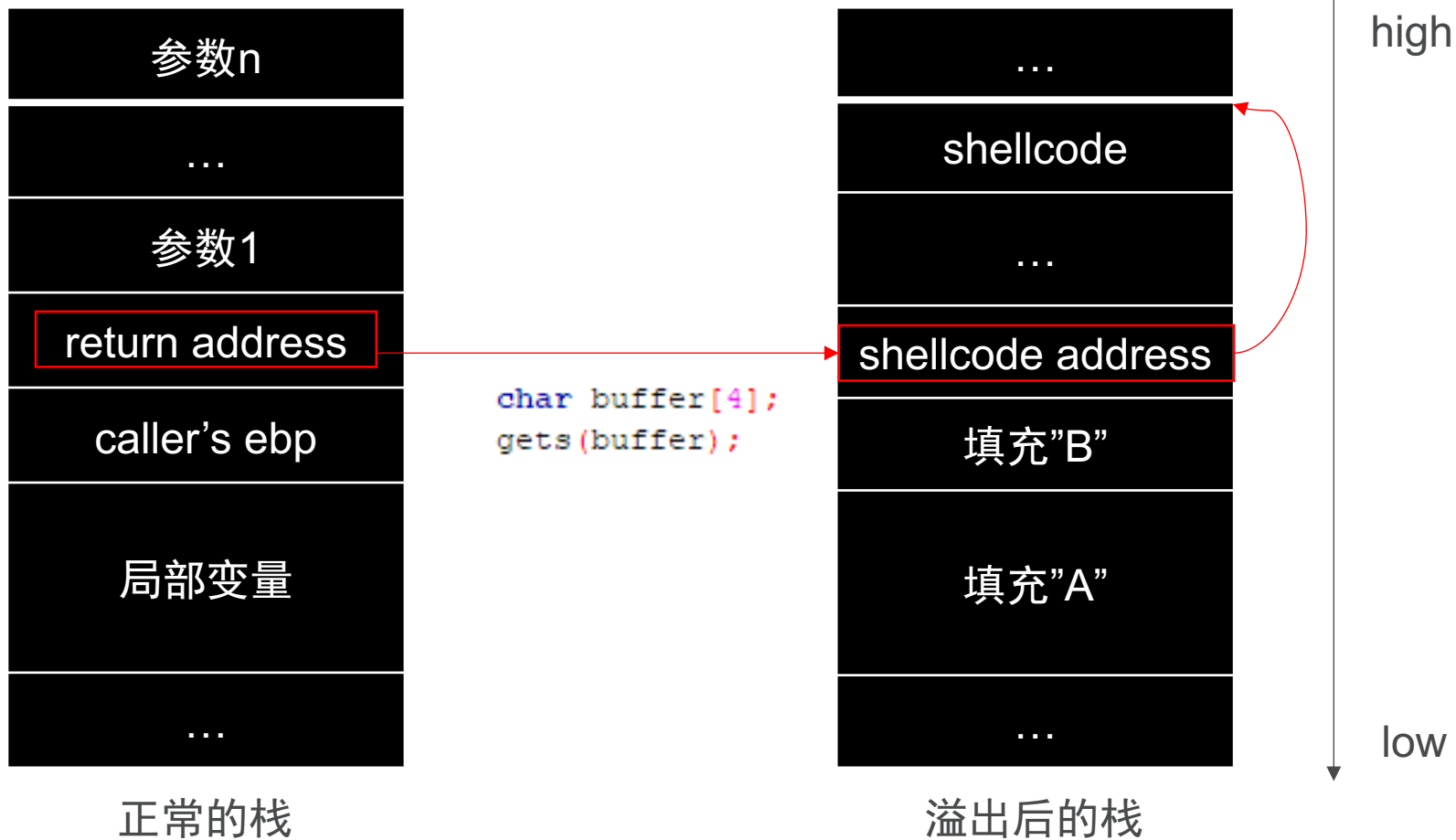
Exploiting Software Vulnerability-Techniques and Practice

[第四次课] 代码重用攻击与防御

概要

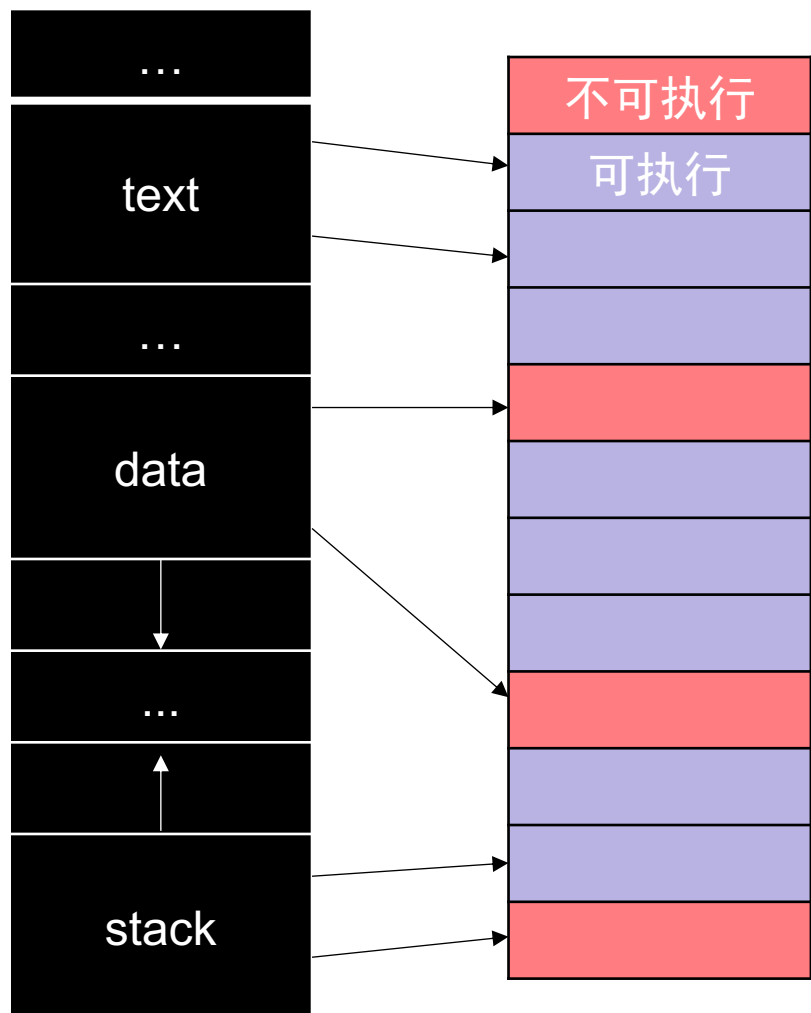
- 回顾栈溢出
- 数据执行保护 (DEP)
- Ret-to-libc
- 地址空间配置随机化 (ASLR)
- 图灵完备
- Return-Oriented Programming (ROP)
- 控制流完整性 (CFI)
- Jump-Oriented Programming (JOP)

栈溢出回顾



数据执行保护（DEP）

- 将原有在内存中无差别存储的内容分为数据段与代码段，并划分了不同的权限：
- **数据不可执行，代码不可修改**
- 使攻击者以溢出缓冲区等形式注入的恶意代码不会再被允许执行，从而抑制了代码注入攻击。
- 实现：
 - 1、硬件（处理器）
 - 2、软件（操作系统）



虚拟内存

物理内存

- AMD：在页面表（page table）中的页面信息加了一个特殊的位，NX位（No eXecute）。
- ① 如果NX位为0，这个页面上可以执行指令。
- ② 如果NX位为1，这个页面上不允许执行指令。如果试图执行指令的话，就会产生异常。
- Intel：在它的CPU上也提供了类似的支持，称为XD位（eXecute Disable），其原理和AMD的NX是完全一样的。
- 操作系统对DEP的支持，就是将系统或应用程序的数据页面，标注上NX位。这样，一旦由于堆栈缓存溢出的安全漏洞，导致恶意代码试图在数据页面上运行，CPU就会产生异常，导致程序终止，而不是去执行恶意指令。

代码重用

○产生的原因

DEP等技术限制了注入到栈上的shellcode的执行

○技术的特点

- ① 不需要注入shellcode
- ② 利用已有的动态链接库或其它可执行文件
- ③ 组合文件中已有的函数或指令片段，实现特定功能
- ④ 能够有效突破DEP技术

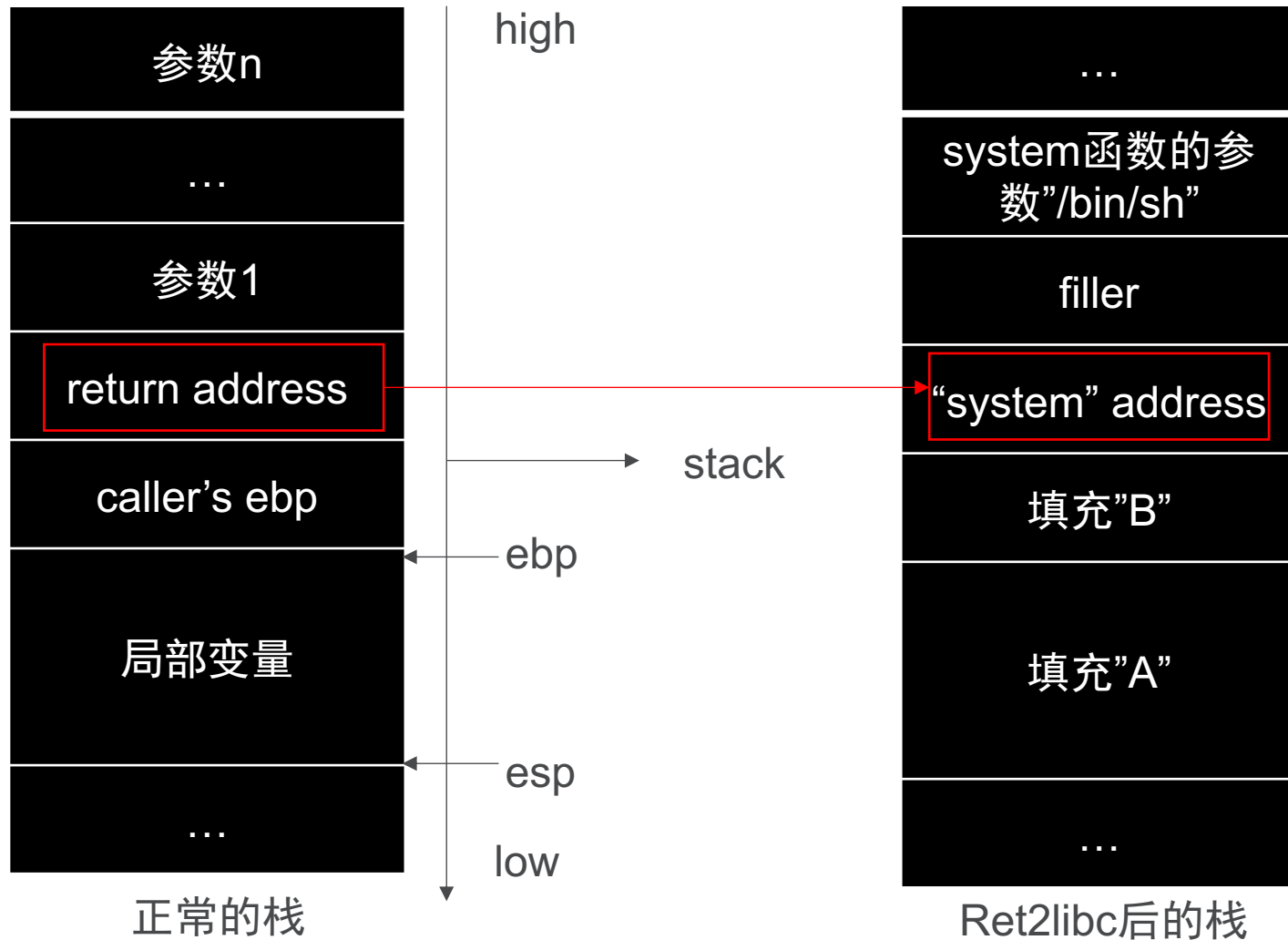
○实现的前提

- ① 存在能够控制栈区的缓冲区溢出漏洞
- ② 程序已有代码和指令的位置可以被找到

Return-to-Libc

- 程序是动态链接生成的，所以在程序运行时会调用 `libc.so`，`libc.so` 中包含了大量的可以利用的函数，包括 `system()`、`execve()` 等系统级函数，通常情况下，我们会选择执行 `system("/bin/sh")` 来打开 shell
- 1、找到 `system()` 函数的地址；
- 2、在内存中找到 `"/bin/sh"` 这个字符串的地址。
- 通过找到这些函数在内存中的地址覆盖掉返回地址来获得当前进程的控制权。

Return-to-Libc图示



地址空间配置随机化 (ASLR)

- 通过对堆、栈、共享库映射等线性区布局的随机化，通过增加攻击者预测目的地址的难度，防止攻击者直接定位攻击代码位置，达到阻止溢出攻击的目的的一种技术。
- 绕过方法：
- 只有libc基地址是随机的，特定功能的偏移与其基地址始终保持不变！
- 只要泄露出动态库中某个函数的地址，就可以知道该函数在动态库中的偏移。
- 动态库的基址=泄露的函数的地址 - 该函数在动态库中的偏移量
- system函数的地址= 动态库的基址 + system函数在动态库中的偏移量

图灵完备

- 在可计算性理论里，如果一系列操作数据的规则（如指令集、编程语言、细胞自动机）可以用来模拟单带图灵机，那么它是图灵完备的。
- 一般认为Return-to-Libc不具有图灵完备性

ROP攻击

ROP : Return-Oriented Programming 面向返回函数的编程

- ROP攻击的特点：
- 不调用系统中已有的函数
- 扫描已有的动态链接库和可执行文件
- 组合程序中已有的ret结尾的短指令片段
- 这些短指令片段一般不超过5条（ Gadget ）

Gadget

怎样构建Gadget ?

- 程序的指令以二进制的形式贮存于计算机中，CPU 则逐条执行这些指令。但是由于指令集编码的缘故，随着解析的起点不一样，同一字节序列可以解析出多种有效的指令序列。

```
f7 c7 07 00 00 00
0f 95 45 c3
```

```
test $0x00000007,%edi
setnzb -61(%ebp)
```

libc的ecb_crypt的函数入口处的指令

```
c7 07 00 00 00 0f
95
45
c3
```

```
movl $0xf000000, (%edi)
xchg %ebp,%eax
inc %ebp
ret
```

Gadget

怎样构建Gadget？

○ **目的**：在libc的代码段中寻找以ret指令结尾的指令序列，并获取这些指令序列的起始地址。

○ **步骤**：

- ① 构造Trie树结构存储指令序列
- ② Ret指令作为根节点
- ③ 递归反向遍历libc库的代码字节序列，节点存储有效指令序列的地址
- ④ 完成遍历，从根节点到树中任意节点的路径构成有效的指令序列
- ⑤ 取路径长度为1的节点，构成长度为2的指令序列.....直到20字节

○ **人工分析/自动化分析Trie中可利用的指令序列**

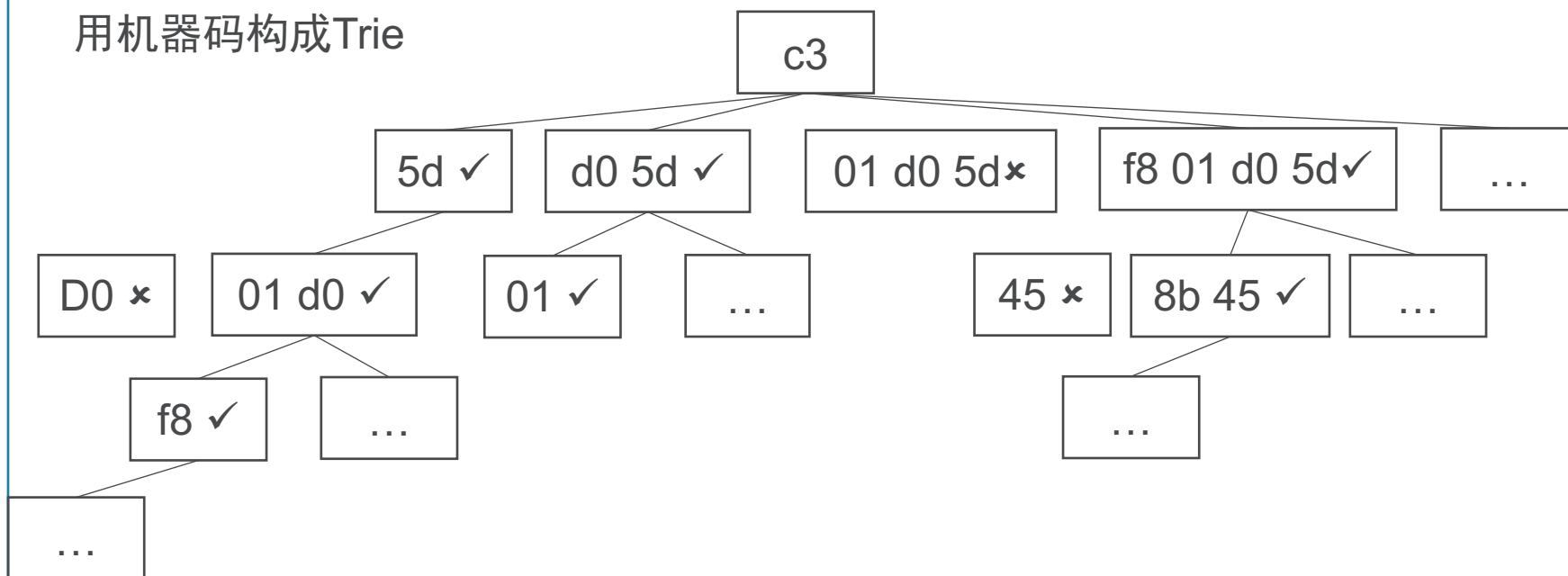
Gadget

怎样构建Gadget ?

假设在libc的代码段中找到如下机器码：

... 55 48 89 e5 89 7d fc 89 75 f8 8b 55 fc 8b 45 f8 01 d0 5d **c3** 23 ...

用机器码构成Trie



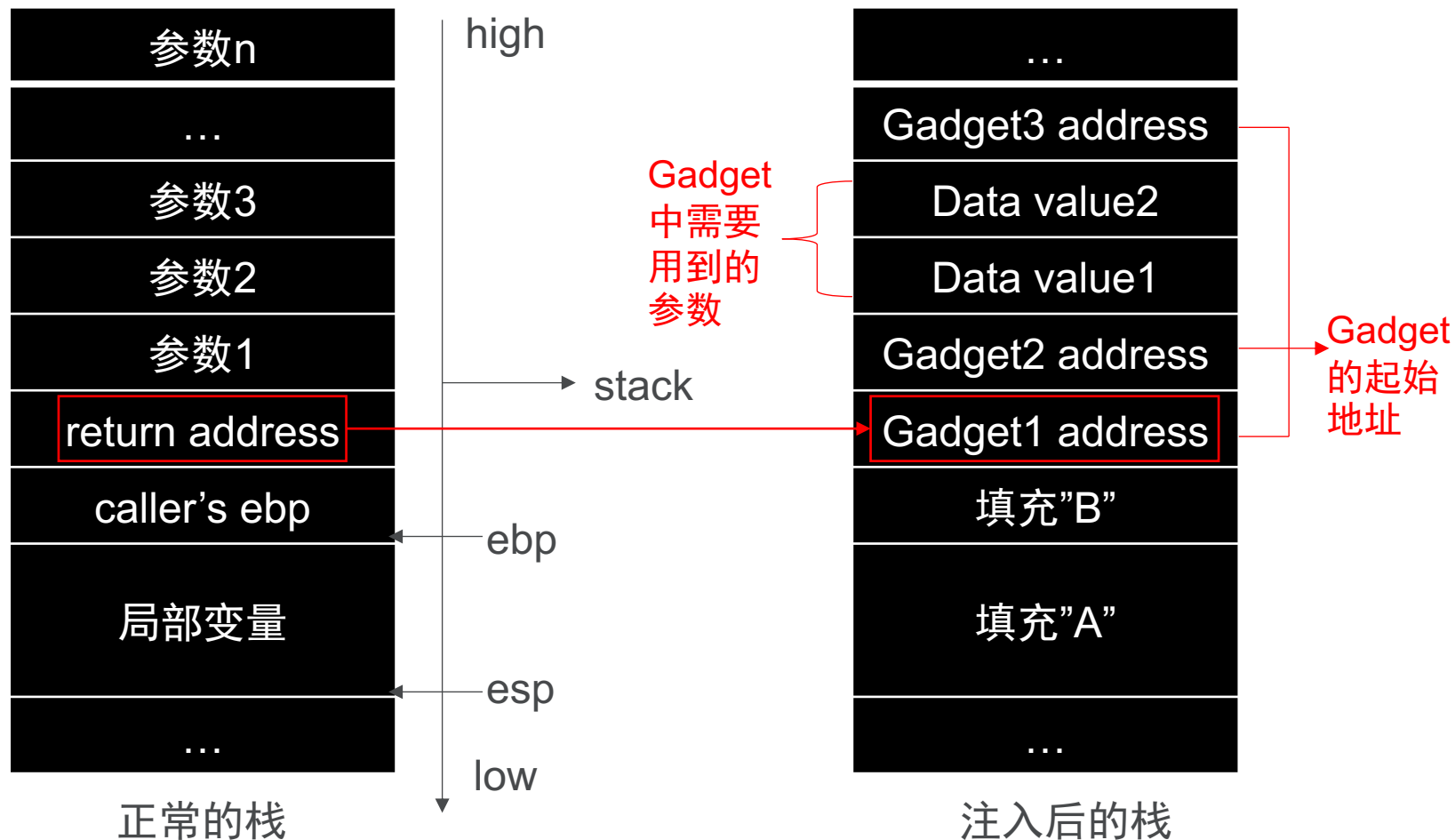
Gadget

如何串联执行Gadget？

- 将Gadget的地址预先注入到栈中
- 劫持控制流,将程序执行流转向第一个ROP Gadget
- 利用Ret指令的特性,跳转到栈顶指向的下一个Gadget

ROP攻击流程

1. 利用缓冲区溢出漏洞将Gadget的地址预先注入到栈中



ROP攻击流程

2. 劫持控制流,将程序执行流转向第一个ROP Gadget

函数结尾, 执行返回指令ret时:

- $eip = [esp]$
- $esp = esp + 1$

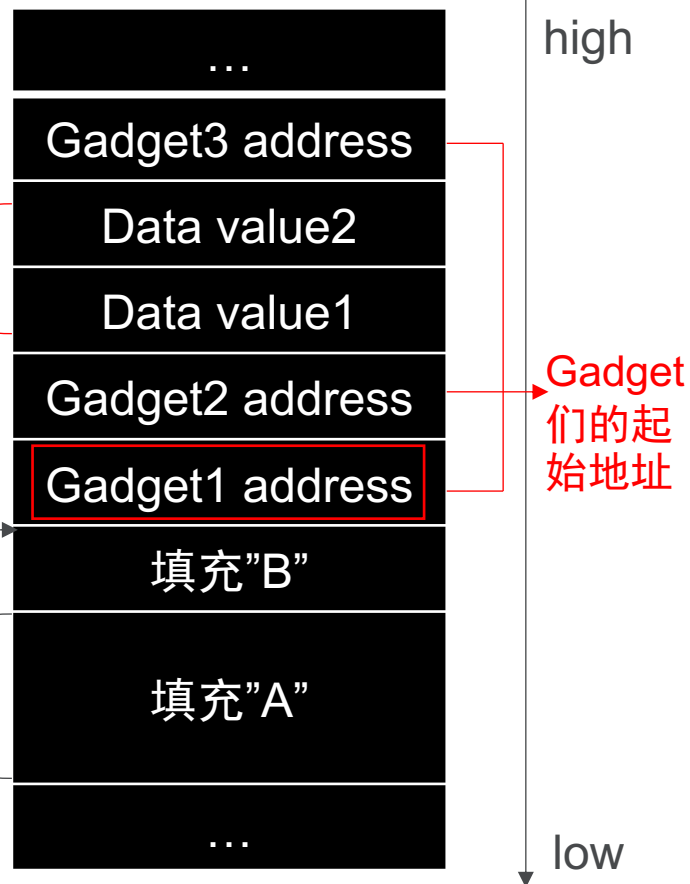
正常情况: 返回到call指令的下一条指令

注入后 : 开始执行第一个Gadget

Gadget
中需要
用到的
参数

esp

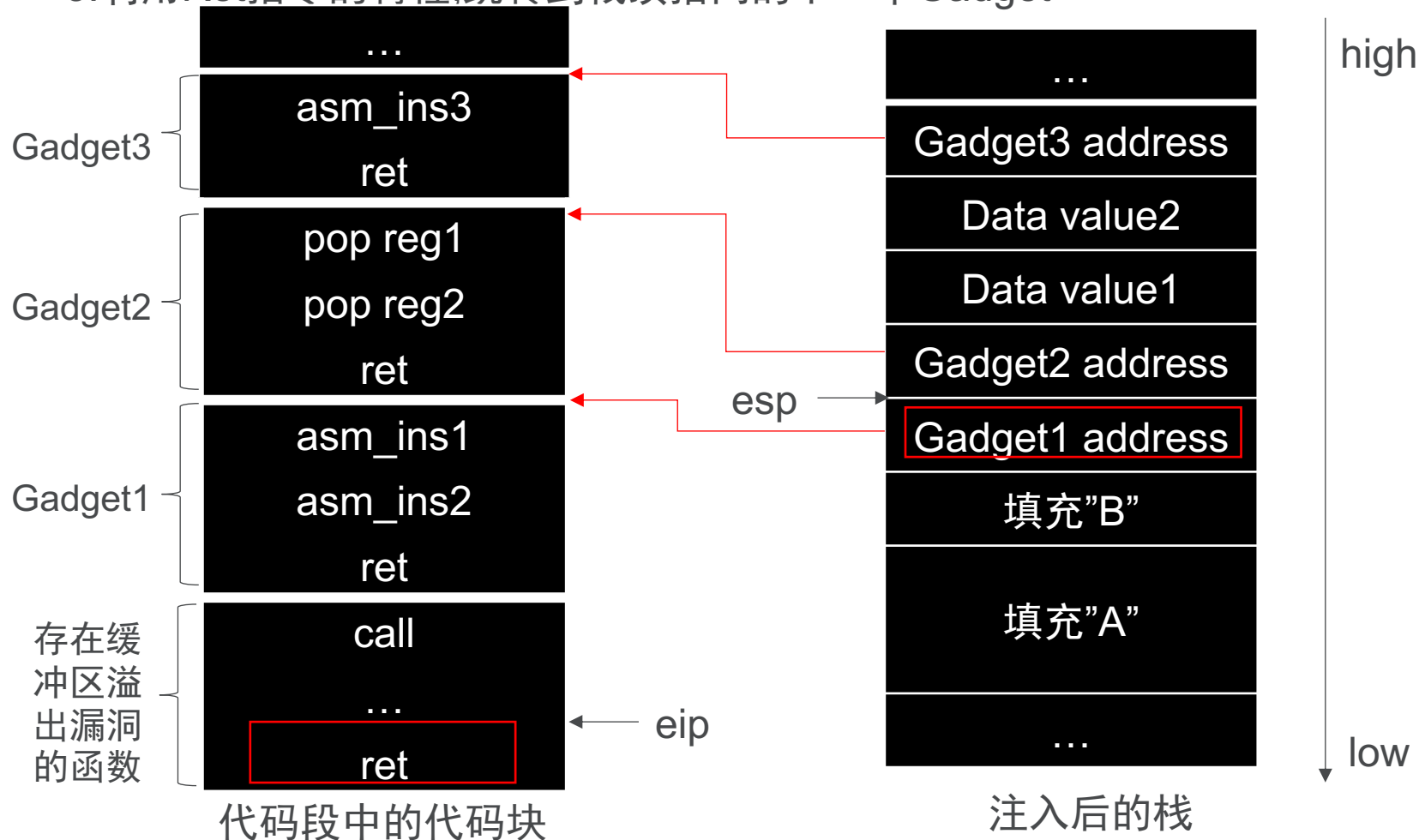
代码执行时所需的
变量值/函数的参数



注入后的栈

ROP攻击流程

3. 利用Ret指令的特性, 跳转到栈顶指向的下一个Gadget



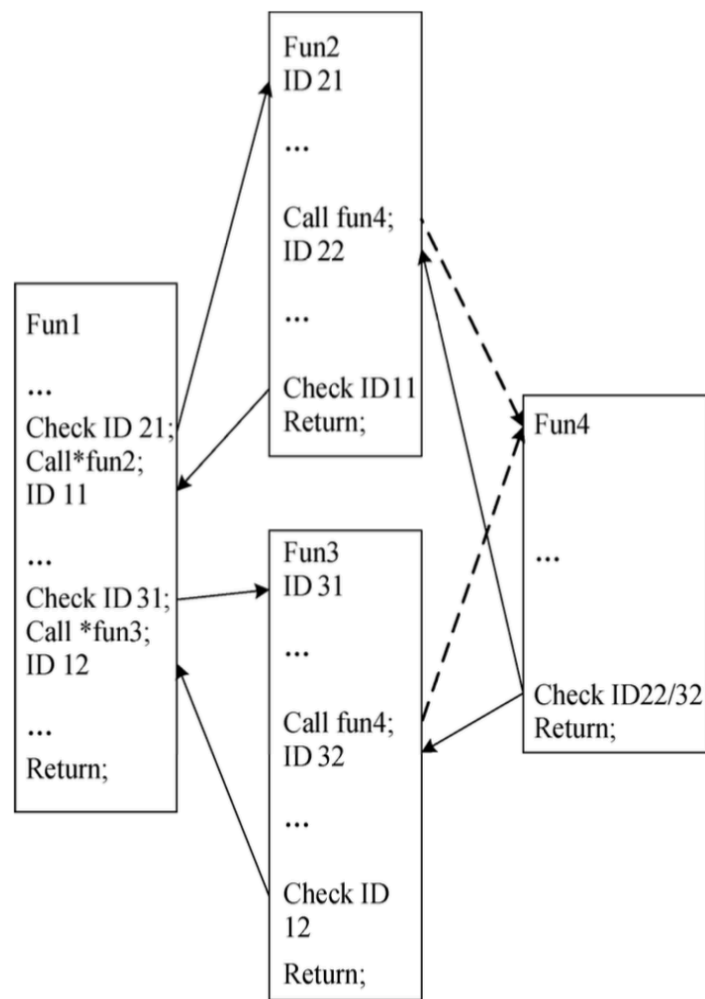
ROP缺陷

ROP攻击有什么缺点？

- 由于ROP 攻击是利用Ret指令在Gadget之间跳转，有很明显的特征，很容易被检测到
- 这些Ret指令没有对应的Call指令，也可以通过检测Call-ret 指令组合发现这类攻击

控制流完整性 (CFI)

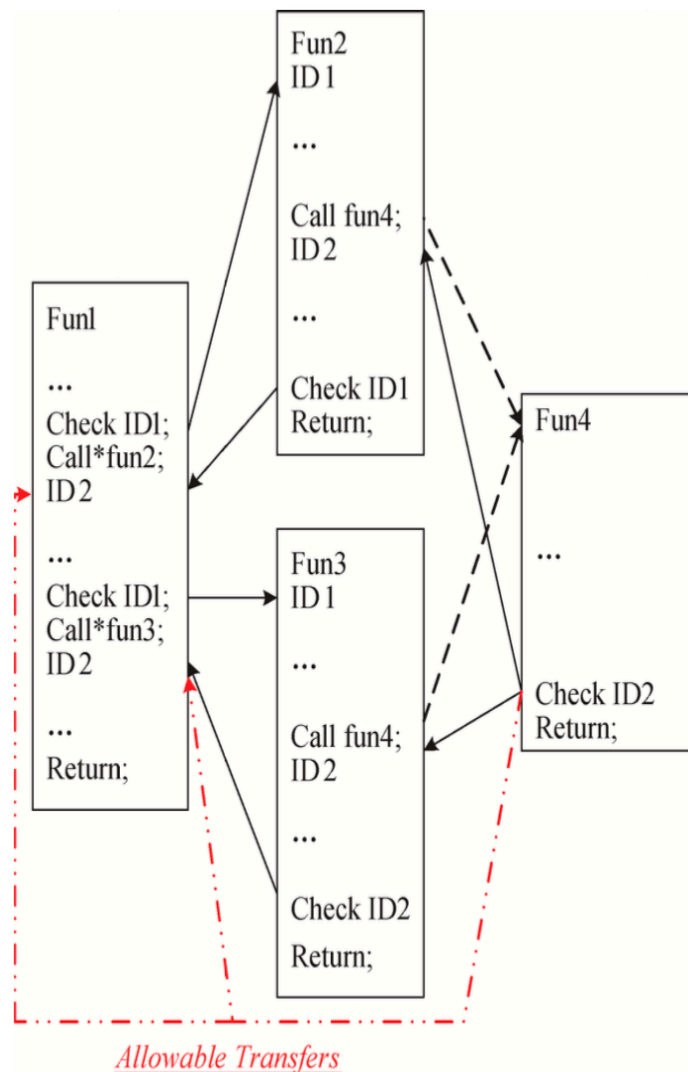
- **细粒度：**
- 将要执行的代码进行静态分析，画出控制流图，预先计算出所有的间接转移指令的可能目标地址，并为其配以唯一ID。
- 改写程序二进制代码，在每一条间接跳转指令执行之前检查逻辑，一旦出现违反控制流图的间接跳转就会报错并终止程序。
- **开销大！**



控制流完整性 (CFI)

○粗粒度：

- 不需要画出程序详细的控制流图，并且对于间接跳转目的地址不需要分配唯一ID。仅对程序进行静态分析，根据一定规则找到合法的跳转地址。如：
- 1、所有的ret指令合法目标都是一条call指令下一条指令；
- 2、所有间接调用目标地址都必须是一个函数的开头。



○可绕过！

JOP攻击

JOP : Jump-Oriented-Programming

- JOP攻击的特点：
- 仍然需要构造指令进行Gadget串联
- 用`jmp`指令替换`ret`指令
- 使用`jmp`作为Gadget的结尾

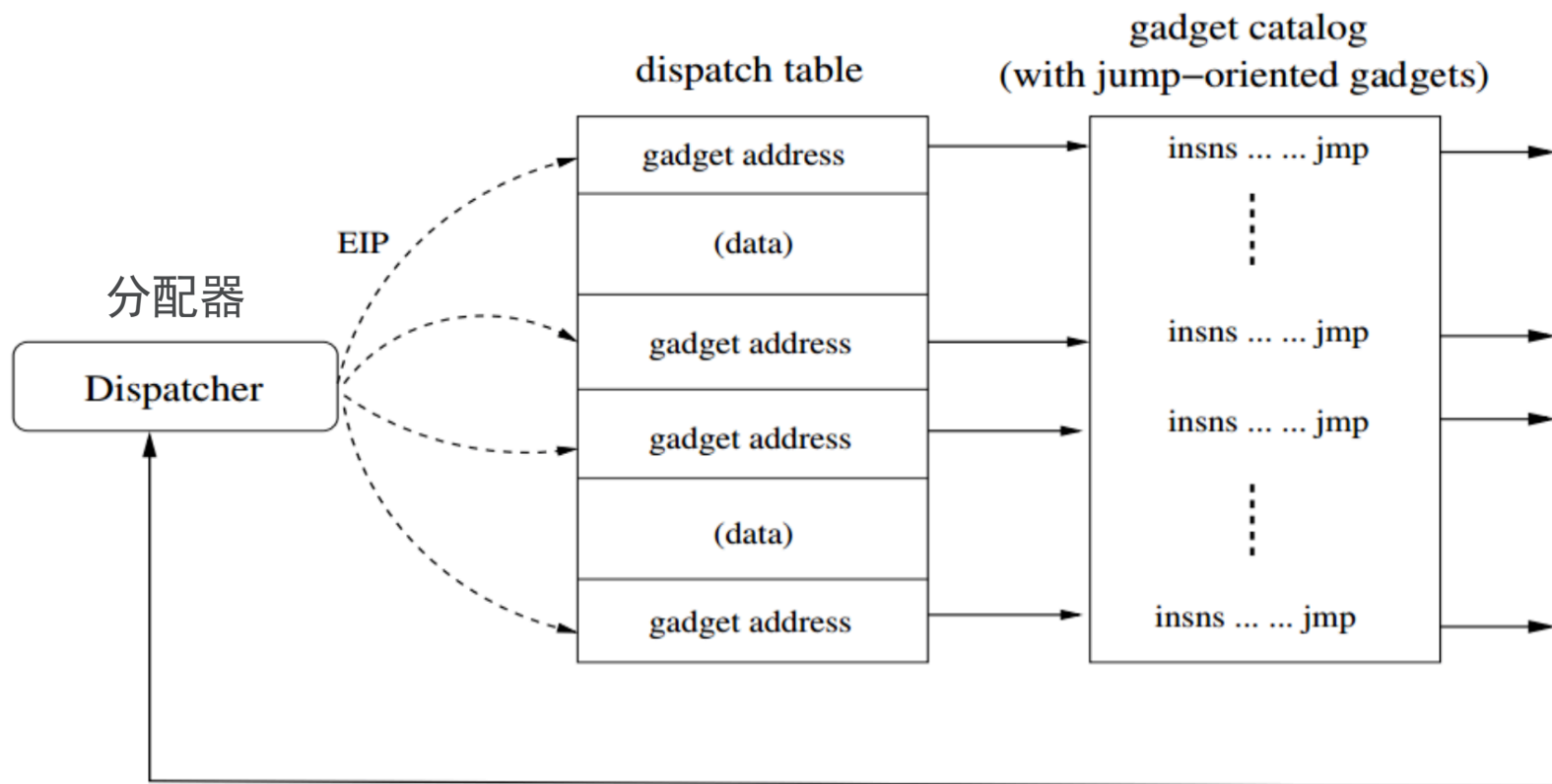
JOP的组成部分

- **Dispatcher Gadget (调度器) :**
 - 用于决定下一步调用哪一个Gadget
- **Dispatch table (调度表)**
 - 用来存储各个Gadget的地址和数据
- **Gadget Catalog**
 - 调度表中各个Gadget包含的指令

JOP攻击示意图

栈中注入分配表

代码段执行Gadget



实验部分——Ret2libc/ROP/JOP

○ 实验环境：

- CPU : Intel(R) Core(TM) i5-3337U CPU @ 1.80GHz
- 主机OS : 4.15.0-45-generic内核Ubuntu 16.04.1 i686

○ ROP Chain 自动化工具：ROPgadget / Ropper

○ 漏洞程序 / Gadgets 指令集

○ 实验目标：执行 `execve("/bin/sh", argv, envp)`

- 通过 `int 80` 执行
- 四个寄存器：`eax` 寄存器传递系统调用号 `0xb`

`ebx` 寄存器传递 `"/bin/sh"` 字符串的地址

`ecx` 寄存器传递参数 `argv`

`edx` 寄存器传递环境变量 `envp`

```
void overflow() {  
    char buf[256];  
    fscanf(fd,"%[^\\n]",buf);  
    return;  
}  
  
int main(int argc, char** argv) {  
    char* filename = "exploit";  
    if(argc>1) filename = argv[1];  
    fd=fopen(filename, "r");  
    overflow();  
}
```

Q&A